

Etapa 2 — Massa de Dados de Treinamento

Autor

Marcus Vinícius Santos de Almeida

Data de entrega

04/09/2026

Objetivo da etapa

Gerar sinteticamente 100+ registros de treinamento, com padrões intencionais e distribuição de classes razoável

Artefatos

`dados/gerar_dados.py` `dados/etapa2_dados_treinamento.csv`
`dados/etapa2_validacao.txt`

1 Metodologia de geração

1.1 Por que dados sintéticos

A Etapa 2 do enunciado exige explicitamente que os 100+ registros sejam **gerados por IA / sinteticamente**, e não coletados de repositórios reais — usar dados reais aqui descumpriria a atividade. A geração é feita por um **script Python** (`dados/gerar_dados.py`), não por texto solto produzido por um chat, para que os padrões sejam programados de forma deliberada, inspecionável e reproduzível.

1.2 Por que programar as correlações de propósito

O Naive Bayes assume independência condicional entre as features. Sabe-se da literatura (Etapa 1) que isso é falso neste domínio. Em vez de esconder o problema, os dados foram construídos para **conter as violações de forma controlada**, alimentando a análise crítica da Etapa 4.

Padrão programado no gerador	Fundamentação
Complexidade ciclomática e LOC nascem correlacionadas (complexidade cresce com o tamanho + ruído próprio)	Shepperd (1988): a complexidade é, em boa parte, proxy de LOC
Subpopulação de arquivos pequenos e densos (LOC baixo + complexidade alta) com risco elevado — não se assume "LOC alto = mais risco" linear	Koru et al. (2008): lei de potência; módulos menores proporcionalmente mais propensos a defeito
Churn gerado como taxa relativa (% do arquivo alterado em 90 dias), nunca valor absoluto	Nagappan & Ball (2005): churn relativo prediz melhor e reduz dependência de tamanho
Nº de imports com correlação apenas leve com LOC, preservando variação própria	Analogia ao achado de que acoplamento (CBO) correlaciona com tamanho — dependência mantida fraca de propósito

Cobertura de testes quase independente das demais features, efeito fraco sobre o rótulo	Inozemtseva & Holmes (2014); Gren & Antinyan (2017): relação cobertura ↔ defeitos é contestada
Ruído controlado em todas as features e no rótulo — nenhuma é sorteio uniforme puro nem regra determinística perfeita	Requisito da atividade: dados "intencionais e realistas"

1.3 Parâmetros principais (ajustáveis no topo do script)

Parâmetro	Valor	Significado
N_REGISTROS	150	Nº de módulos gerados (mínimo exigido: 100)
SEED	42	Semente única para todos os geradores aleatórios — garante reprodutibilidade
PROPORCAO_DEFEITO	0,35	Fração de módulos com <code>defeito = SIM</code> (longe de 0,5 e dos extremos)

1.4 Modelo generativo (resumo)

LOC vem de uma log-normal (muitos arquivos pequenos, poucos grandes). **Complexidade** é uma tendência linear sobre $\text{LOC} \times \text{ruído próprio}$, mais uma subpopulação forçada de arquivos curtos com complexidade "alta". **Nº de autores** é uma contagem de Poisson com média levemente ligada ao tamanho. **Churn relativo** é log-normal em torno de ~10,5%, independente do tamanho absoluto. **Nº de imports** tem tendência fraca sobre LOC somada a ruído forte e a um termo de Poisson independente. **Cobertura** é normal em torno de 63%, com leve queda quando o churn é alto. O **rótulo** é a soma de pesos (log-odds) por categoria + bônus da interação "pequeno e denso" + ruído gaussiano; os $k = \text{round}(N \times 0,35)$ módulos de maior score recebem SIM — a proporção fica exata e o rótulo continua estocástico. Cada feature é salva em **duas formas**: valor bruto e categoria (baixo/médio/alto), nas mesmas faixas da Etapa 1.

2 Resultados da validação

Reproduzível com `python3 dados/gerar_dados.py` ; saída completa em `dados/etapa2_validacao.txt` .

2.1 Proporção de classes

Classe	Registros	Proporção
SIM (com defeito)	52	34,7%
NÃO (sem defeito)	98	65,3%

Distribuição desbalanceada de forma moderada e proposital (~1:2), próxima do `PROPORCAO_DEFEITO = 0,35` e longe de extremos.

2.2 Matriz de correlação de Pearson (features numéricas)

	Complex.	LOC	Autores	Churn	Imports	Cobert.
Complex.	1,00	0,67	0,19	-0,08	0,12	0,11
LOC	0,67	1,00	0,29	-0,11	0,20	-0,04
Autores	0,19	0,29	1,00	-0,10	0,02	0,09
Churn	-0,08	-0,11	-0,10	1,00	0,16	-0,08
Imports	0,12	0,20	0,02	0,16	1,00	-0,11
Cobert.	0,11	-0,04	0,09	-0,08	-0,11	1,00

Leitura da matriz

- **Complexidade × LOC = 0,67** — de longe a maior correlação da matriz, confirmando que a dependência programada de propósito (Shepperd, 1988) realmente aparece nos dados. Será o exemplo central da discussão de violação de independência na Etapa 4.
- **LOC × Nº de imports = 0,20** — correlação leve, como pretendido.
- **LOC × Churn relativo = -0,11** — praticamente nula, confirmando que o churn foi gerado como taxa e não herdou a escala de tamanho.
- Todas as demais correlações têm $|r| < 0,30$.

2.3 Estatísticas descritivas (valor bruto de cada feature)

Feature	Média	Desvio	Mín.	Mediana	Máx.
Complexidade ciclomática	17,58	20,56	1	10,0	120
LOC	122,22	146,15	12	87,5	1434
Nº de autores	2,40	1,26	1	2,0	7
Churn relativo (%)	13,52	11,68	0,7	10,5	85,0
Nº de imports	6,31	3,54	1	5,5	22
Cobertura de testes (%)	61,13	18,65	15,6	63,5	99,0

As distribuições de LOC e complexidade são fortemente assimétricas à direita (média $\gg$ mediana), como esperado para métricas de tamanho de código.

2.4 Checagem dos padrões programados

Taxa de defeito $P(SIM)$ por categoria de cada feature — taxa geral = 34,7%:

Feature	Baixo	Médio	Alto
LOC	38%	30%	50%
Complexidade ciclomática	15%	38%	70%

Churn relativo	29%	38%	46%
Nº de autores	24%	35%	52%
Nº de imports	24%	32%	55%
Cobertura de testes	29%	39%	32%

Padrões confirmados na saída

- **Complexidade, autores, imports e churn:** taxa de defeito cresce de forma monótona com a categoria — sinal preditivo claro.
- **LOC:** relação **não monótona** (baixo 38%, médio 30%, alto 50%). Arquivos pequenos são mais arriscados que os médios — exatamente o comportamento contraintuitivo de Koru et al. antecipado na Etapa 1.
- **Cobertura de testes:** praticamente sem sinal (29% / 39% / 32%) — reproduz de propósito a fraqueza documentada dessa feature.
- **Subpopulação "pequeno e denso"** (LOC baixo + complexidade alta): 8 módulos, **todos rotulados SIM** (taxa de defeito = 100% vs. 34,7% geral) — grupo pequeno e deliberadamente extremo, construído para representar com força o achado central de Koru et al.

3 Conformidade com os requisitos da atividade

Requisito (Etapa 2 do enunciado)	Situação
Pelo menos 100 registros	150 registros (<code>N_REGISTROS = 150</code>)
Padrões intencionais e realistas, não aleatórios	Correlações e efeitos programados no gerador e confirmados na validação (§2.2 e §2.4)
Distribuição razoável entre classes	34,7% / 65,3% ($\approx 1:2$)
Usar apenas as features discretizadas da Etapa 1	6 features, mesmas faixas baixo/médio/alto; CSV traz valor bruto e categoria
Dados salvos em tabela incluída no entregável	<code>dados/etapa2_dados_treinamento.csv</code> (150 linhas + cabeçalho)

Reprodutibilidade

Rodar `python3 dados/gerar_dados.py` regenera exatamente o mesmo CSV e o mesmo relatório de validação (semente fixa `SEED = 42`). Os parâmetros no topo do script permitem ajustar a massa sem tocar na lógica.

Nota sobre o uso de IA

O script, os parâmetros do modelo generativo e este relatório foram construídos em diálogo com uma IA generativa; os padrões estatísticos programados foram derivados das referências verificadas na Etapa 1 (Shepperd, 1988; Koru et al., 2008; Nagappan & Ball, 2005; Inozemtseva & Holmes, 2014; Gren & Antinyan, 2017) e **conferidos na saída de validação** antes de fixar os valores. Todo o código é comentado e defensável linha a linha.